```
    def choose_arm(self, arm):                          2
        reward = self.get_reward(arm)
        self.update_state()
        return reward
```

- ***1*** Number of states = number of arms, to keep things simple. Each row represents a state and each column an arm.
- ***2*** Choosing an arm returns a reward and updates the state.

The following code snippet demonstrates how to use the environment. The only part we need to build is the agent, which is generally the crux of any RL problem, since building an environment usually just involves setting up input/output with some data source or plugging into an existing API.

```
env = ContextBandit(arms=10)
state = env.get_state()
reward = env.choose_arm(1)
print(state)
>>> 2
print(reward)
>>> 8
```

The simulator consists of a simple Python class called `ContextBandit` that can be initialized to a specific number of arms. For simplicity, the number of states equals the number of arms, but in general the state space is often much larger than the action space. The class has two methods: One is `get_state()`, which is called with no arguments and will return a state sampled randomly from a uniform distribution. In most problems your state will come from a much more complex distribution. Calling the other method, `choose_arm(...)`, will simulate placing an advertisement, and it returns a reward (e.g., proportional to the number of ad clicks). We need to always call `get_state` and then `choose_arm`, in that order, to continually get new data to learn from.

The `ContextBandit` module also includes a few helper functions, such as the `softmax` function and a *one-hot encoder*. A one-hot encoded vector is a vector where all but 1 element is set to 0. The only nonzero element is set to 1 and indicates a particular state in the state space.